

Le concept de **MapReduce** (ou map-reduce) n'a rien de nouveau : il s'inspire grandement des principes de base de la programmation fonctionnelle (c'est-à-dire, par usage de fonctions mathématiques).

Comme son nom l'indique, map-reduce se compose de deux fonctions :

- **map**, qui passe à reduce une clé et des données à traiter
- **reduce** qui, pour chaque clé donnée par map, va opérer une réduction sur les données en provenance de map selon une logique que vous allez devoir écrire.

Map-reduce a connu un renouveau avec l'apparition des data stores de très grande capacité sur le web et doit en grande partie son retour sous les feux de la rampe à Google qui s'en sert pour distribuer/paralléliser les traitements, sur des clusters de machines. Le principe est simple : découper un gros problème en plusieurs petits, résolus par distribution à des nœuds fils (réducteurs) qui ensuite remontent le résultat de ces opérations aux nœuds de niveau supérieur, et ce récursivement jusqu'à remonter au la racine de l'arbre, qui a initié le calcul et en récupère donc le résultat final.

Plus de détails sur le MapReduce : <https://fr.wikipedia.org/wiki/MapReduce>

Créer une nouvelle base de données `bdcommandes`, puis la collection `commandes`.
use `bdcommandes`

Exécuter le script suivant (**fichier commandes.js**) :

```
var cdes = [
{
  "user":
  {
    "id": 54847,
    "nom": "DUPONT Jean",
    "sexe": "M",
    "cp": 13000
  },
  "games": [
    {
      "id": 76,
      "titre": "NBA 2K24 Edition Légende Black Mamba",
      "prix": 74.99
    },
    {
      "id": 2,
      "titre": "Super Mario Bros. Wonder Nintendo Switch",
      "prix": 49.99
    }
  ],
  "tva": 25.00,
  "totalht": 124.98,
  "date": ISODate("2024-10-20T00:00:00Z")
},
{
  "user":
  {
    "id": 54846,
    "nom": "DUPONT Marie",
    "sexe": "F",
    "cp": 13000
  },
  "date": ISODate("2024-10-21T00:00:00Z"),
  "games": [
    {
      "id": 76,
      "titre": "NBA 2K24 Edition Légende Black Mamba",
      "prix": 74.99
    },
    {
      "id": 3,
      "titre": "Marvel's Spider-Man 2",
      "prix": 69.99
    }
  ],
  "tva": 29.00,
  "totalht": 144.98
},
]
```

```

{
  "user":
    {
      "id": 54845,
      "nom": "COLIN Pascal",
      "sexe": "m",
      "cp": 74000
    },
  "date": ISODate("2024-09-20T00:00:00Z"),
  "games": [
    {
      "id": 3,
      "titre": "Marvel's Spider-Man 2",
      "prix": 69.99
    }
  ],
  "tva": 14.00,
  "totalht": 69.99
},
{
  "user":
    {
      "id": 54843,
      "nom": "COUDRE Vince",
      "sexe": "m",
      "cp": 74000
    },
  "date": ISODate("2024-10-10T00:00:00Z"),
  "games": [
    {
      "id": 1,
      "titre": "EA Sports FC 24",
      "prix": 39.00
    }
  ],
  "tva": 7.80,
  "totalht": 39.00
},
{
  "user":
    {
      "id": 54842,
      "nom": "COUDRE Pauline",
      "sexe": "f",
      "cp": 74000
    },
  "date": ISODate("2024-10-22T00:00:00Z"),
  "games": [
    {
      "id": 10,
      "titre": "Dragon Ball: Sparking! Zero",
      "prix": 66.99
    },
    {
      "id": 11,
      "titre": "Call of Duty : Black Ops 6",
      "prix": 69.99
    }
  ],
  "tva": 27.40,
  "totalht": 136.98
},
{
  "user":
    {
      "id": 54842,
      "nom": "COUDRE Elodie",
      "sexe": "F",
      "cp": 74000
    },
  "date": ISODate("2024-10-22T00:00:00Z"),
  "games": [
    {
      "id": 1,
      "titre": "EA Sports FC 24",
      "prix": 39.00
    },
    {
      "id": 11,
      "titre": "Call of Duty : Black Ops 6",
      "prix": 69.99
    }
  ],
  "tva": 27.40,
  "totalht": 136.98
}

```

```

        "id": 3,
        "titre": "Marvel's Spider-Man 2",
        "prix": 69.99
    },
    ],
    "tva": 35.80,
    "totalht": 178.98
},
{
    "user":
    {
        "id": 54841,
        "nom": "HENRY Fanny",
        "sexe": "f",
        "cp": 69001
    },
    "date": ISODate("2024-09-01T00:00:00Z"),
    "tva": 40.00,
    "totalht": 200.00
},
{
    "user":
    {
        "id": 54840,
        "nom": "HENRY Mathieu",
        "sexe": "M",
        "cp": 69001
    },
    "date": ISODate("2024-09-01T00:00:00Z"),
    "tva": 20.00,
    "totalht": 100.00
}
]
db.commandes.insertMany(cdes)

```

```

'0': ObjectId("65278a41593f9887a401dc2d"),
'1': ObjectId("65278a41593f9887a401dc2e"),
'2': ObjectId("65278a41593f9887a401dc2f"),
'3': ObjectId("65278a41593f9887a401dc30"),
'4': ObjectId("65278a41593f9887a401dc31"),
'5': ObjectId("65278a41593f9887a401dc32"),
'6': ObjectId("65278a41593f9887a401dc33"),
'7': ObjectId("65278a41593f9887a401dc34")
}
}
bdcommandes > |

```

Map traverse tous les documents de la collection MongoDB, ici `commandes`, et va pousser des infos à **Reduce** selon une clé définie.

La fonction `mapReduce` de MongoDB (<https://www.mongodb.com/docs/manual/reference/method/db.collection.mapReduce/>) prend donc pour paramètres :

- `map()` :
 - la fonction ne prend aucun paramètre, on accède à l'objet analysé via l'opérateur `this`
 - la fonction peut émettre des couples via la fonction `emit(key, value)` autant de fois que nécessaire dans la fonction
- `reduce()` :
 - la fonction prendre deux paramètres `key` et `values` (tableau des valeurs de la clé)
 - elle peut être appelée plusieurs fois pour la même clé, elle doit donc renvoyer une valeur de même type que celles dans le tableau
- 3^{ème} paramètre : un littéral JSON, qui représente les options de la fonction. La principale option (out) est la collection dans laquelle le résultat sera placé. Si l'on veut voir le résultat, sans le stocker, il est possible d'indiquer `out: { inline: 1 }`

Par exemple pour calculer le nombre d'hommes et de femmes ayant passé une commande :
 Tout d'abord, on définit la fonction `map()`, qui émettra un couple (`sexe, 1`), pour chaque commande.

```
var map1 = function() {
    emit(this.user.sexe, 1)
};
```

Ensuite, nous définissons la fonction `reduce()`, qui fera la somme des valeurs (qui seront tous des 1 donc).

```
var red1 = function(key, values) {
    return Array.sum(values);
};
```

Ensuite, nous faisons appel à la fonction `mapReduce()` sur notre collection.

```
db.commandes.mapReduce(
    map1,
    red1,
    { out: { inline: 1 } }
)
```

Ce code fonctionnait parfaitement jusqu'en V4 de MongoDB. Si vous exécutez ce code sur la version actuelle, le serveur se bloquera. Vous devrez donc arrêter le client Compass puis le relancer.

En effet, depuis la V5, la fonction `mapReduce` est devenue obsolète et, surtout, ne s'exécute plus. Il est nécessaire d'utiliser l'agrégation. Tout est expliqué ici : <https://www.mongodb.com/docs/manual/reference/method/db.collection.mapReduce/>

Ce qui est vraiment dommage, car cela ne permet plus de comprendre le principe du MapReduce utilisé dans de nombreux outils NoSQL, comme, par exemple, Hadoop.

Le code devient ainsi :

```
db.commandes.aggregate([
    { $group: { _id: "$user.sexe", value: { $sum: 1 } } },
    { $out: "nb_sexe" }
])
db.nb_sexe.find().sort( { _id: 1 } )
```

```
> db.commandes.aggregate([
    { $group: { _id: "$userid.sexe", value: { $sum: 1 } } },
    { $out: "nb_sexe" }
])

db.nb_sexe.find().sort( { _id: 1 } )
< {
  _id: 'F',
  value: 2
}
{
  _id: 'M',
  value: 2
}
{
  _id: 'f',
  value: 2
}
{
  _id: 'm',
  value: 2
}
```

On remarque qu'on a « m » et « M ».

Code avec la fonction `mapReduce` (V4) :

On peut résoudre ce problème en faisant une transformation (en majuscule ou en minuscule) dans la fonction `map()`.

```
var map1 = function() {
    emit(this.user.sexe.toUpperCase(), 1)
}
```

On peut aussi tout déclarer dans la fonction `mapReduce()`, en utilisant le principe des fonctions anonymes Javascript :

```
db.commandes.mapReduce(  
  function() {  
    emit(this.user.sexe.toUpperCase, 1)  
  },  
  function(key, values) {  
    return Array.sum(values);  
  },  
  { out: { inline: 1 } }  
)
```

Le code à partir de la V5 devient avec `aggregate` :

```
db.commandes.aggregate([  
  { $group: { _id: { $toUpper: "$user.sexe" }, value: { $sum: 1 } } },  
  { $out: "nb_sexe" }  
)  
db.nb_sexe.find().sort( { _id: 1 } )
```

```
> db.commandes.aggregate([  
  { $group: { _id: { $toUpper: "$userid.sexe" }, value: { $sum: 1 } } },  
  { $out: "nb_sexe" }  
)  
db.nb_sexe.find().sort( { _id: 1 } )  
< {  
  _id: 'F',  
  value: 4  
}  
{  
  _id: 'M',  
  value: 4  
}
```

Voici le code permettant de calculer le CA HT par code postal :

V4 :

```
map = function() {  
  emit(this['user.cp'], {totalht: this['totalht']});  
};  
  
reduce = function(cle, valeur) {  
  var s = {somme : 0};  
  valeur.forEach(function(cde) {  
    s.somme += cde.totalht;  
  });  
  return s;  
};  
  
db.commandes.mapReduce(map, reduce, {out: 'ca_ht_par_cp'});  
  
db.ca_ht_par_cp.find();
```

La fonction **map** va émettre (*emit*) des paires clé/valeur sur lesquelles **reduce** va travailler. Pour faire simple, **reduce** va faire la somme de tous les montants HT des commandes pour un code postal donné. Une collection est créée pour stocker le résultat de l'exécution de map-reduce : nous l'avons nommée `ca_ht_par_cp`.

>=V5 :

```
db.commandes.aggregate([  
  { $group: { _id: "$user.cp", value: { $sum: "$totalht" } } },
```

```

    { $out: "ca_ht_par_cp" }
  })
  db.ca_ht_par_cp.find().sort( { _id: 1 } )

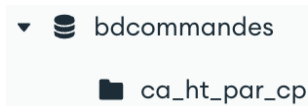
```

```

> db.commandes.aggregate([
  { $group: { _id: "$userid.codepost", value: { $sum: "$totalht" } } },
  { $out: "ca_ht_par_cp" }
])
db.ca_ht_par_cp.find().sort( { _id: 1 } )
< {
  _id: 13000,
  value: 1091.3000000000002
}
{
  _id: 69001,
  value: 300
}
{
  _id: 74000,
  value: 4693
}

```

Collection créée :



Elle est persistante, elle ne sera pas détruite une fois que vous vous déconnectez.

Vous réaliserez des requêtes utilisant le principe du MapReduce en utilisant la fonction `agregate`. Exemples de requêtes à écrire :

- Calculer le CA HT par sexe
- Calculer le CA TTC par CP
- Calculer le nombre de commandes par nom client (`user.nom`)
- Calculer le nombre de commandes par client de la même famille (même nom de famille, par ex. DUPONT)
- Calculer le nombre de jeux commandés par nom client
- Calculer le CA par titre de jeu
- Calculer le CA par année/mois
- Calculer le nombre moyen de jeux par commande
- Etc.

A chaque fois vous indiquerez l'intitulé de la requête réalisée, le code de la requête, ainsi que la copie d'écran du résultat.
